SCSE3203 – Special Topics

WEEK 02 | GUIDED LABORATORY WORKSHEET

Setup → First Server-Rendered Java Web Application

Faculty of Computing, Universiti Teknologi Malaysia • Academic Session 2026/2027

Student: _____ Matric no.: _____ Section: _____

Lab context: Windows 10/11 computer with VS Code installed but no Java development tools configured. This worksheet is the hands-on companion to the approved Week 02 lecture slides and comprehensive master notes.

Laboratory outcomes and constraints

- Configure Java 21 and the required VS Code extensions from a clean installation.
- Use Maven Wrapper to generate, build, test and launch a Spring Boot project.
- Build a server-rendered course website using Spring MVC @Controller and Thymeleaf templates.
- Verify the browser output, diagnose common failures and save the project using Git.

Not included this week: REST API, @RestController, JSON endpoints, Fetch, database, authentication, Thymeleaf form binding or advanced MVC.

Time and priority plan

Stage	Time guide	Priority
Preflight + JDK	35 min	Mandatory
VS Code + Maven/project	40 min	Mandatory
First page/controller + verification	60 min	Mandatory
Second page + configuration	35 min	Target
Build/JAR + Git/GitHub	50 min	Target; finish as SDL if needed
Total	220 min + 20 min troubleshooting buffer	4-hour session

DO NOT skip verification checkpoints. If a checkpoint fails, stop and use the corresponding troubleshooting guidance before proceeding.

A. Preflight: start from VS Code only

1. Confirm Windows 10/11, VS Code installed, internet access, and a standard user folder where you can save projects. Avoid a folder requiring administrator permission.
2. Create a working folder such as C:\SCSE3203\. Ensure adequate disk space for the JDK, downloaded Maven dependencies and build output.
3. Close old terminal windows after any installation that changes PATH. Use a fresh PowerShell window for verification.

VERIFY ☐ VS Code opens; an internet connection is available; you can create a working directory.

B. Install and verify Java 21

Task B1 – Download and install Eclipse Temurin JDK 21

4. Open https://adoptium.net/ and select Eclipse Temurin 21 (LTS), Windows, x64 (if your machine is x64), JDK—not JRE. Confirm the architecture matches your device.
5. Download the Windows MSI installer from the official site. Launch it and follow the installer; where offered, select options to set JAVA_HOME and add Java to PATH.
6. Finish installation. Close and reopen PowerShell (or restart VS Code if its terminal remained open).

Task B2 – Verify installation

```
java -version
javac -version
where.exe java
where.exe javac
```

Expected output:

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>java -version
java version "21.0.12" 2026-07-21 LTS
Java(TM) SE Runtime Environment (build 21.0.12+7-LTS-205)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.12+7-LTS-205, mixed mode, sharing)

C:\Users\User>javac -version
javac 21.0.12

C:\Users\User>where.exe java
C:\Program Files\Common Files\Oracle\Java\javapath\java.exe

C:\Users\User>where.exe javac
C:\Program Files\Common Files\Oracle\Java\javapath\javac.exe


C:\Users\User>
```

Expected: both version commands report Java 21. The where.exe commands return paths to the Java runtime and compiler. The exact build number and install directory can differ.

VERIFY ☐ Both java and javac execute and show major version 21. Record your version: _____.

IF IT FAILS "java is not recognized": open a NEW PowerShell terminal, verify installed JDK and PATH. "javac is not recognized": ensure you installed the JDK, not a JRE. Wrong version: inspect where.exe output, JAVA_HOME and PATH ordering. Do not randomly delete installed JDKs.

Task B3 – Understand the environment

JDK includes tools for Java development; javac compiles source into bytecode; java launches JVM execution. JAVA_HOME should point to the JDK root, while PATH should make its bin directory accessible.

7. In PowerShell, optionally check $env:JAVA_HOME to identify the configured JDK location.

echo %JAVA_HOME% for windows cmd

Expected output:

C:\Users\User>echo %JAVA_HOME%
C:\Program Files\Java\jdk-21.0.12

C:\Users\User>

C. Turn VS Code into a Java development environment

Task C1 – Install required extensions

8. Open VS Code → Extensions (Ctrl+Shift+X). Search and install "Extension Pack for Java" (Microsoft). Do not need to reinstall another JDK when asked
9. Search and install "Spring Boot Extension Pack" (VMware). Wait for extension installation and Java language services to initialise.

VERIFY ☐ Both extension packs appear as installed and VS Code detects a Java 21 JDK.

IF IT FAILS If Java is not detected, reload the VS Code window, check Java configuration and JAVA_HOME, and restart VS Code. If extension download is blocked, report the network/proxy error to the lecturer.

Task C2 – Confirm VS Code tools

10. Open Terminal → New Terminal. Run java -version again inside VS Code. Compare it with the standalone PowerShell result.

```
java -version
```

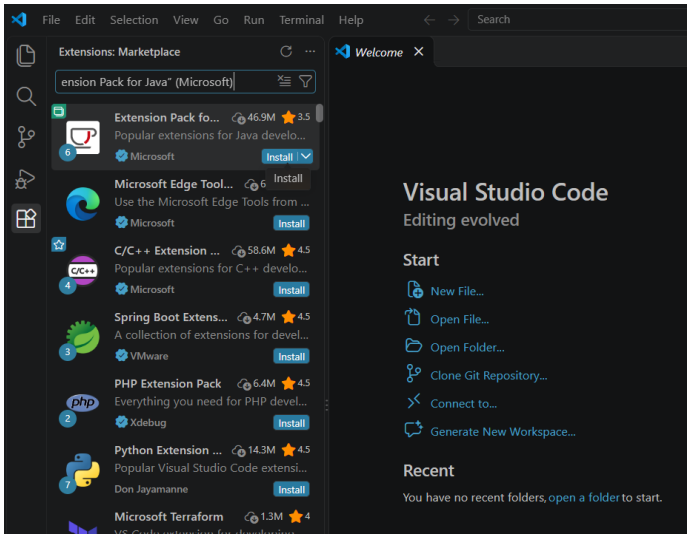
VERIFY ☐ The VS Code terminal reports the same intended Java 21 installation.

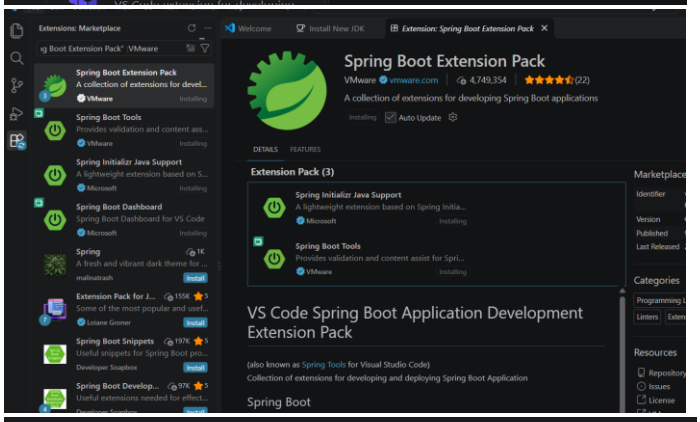
D. Understand Maven Wrapper and embedded Tomcat

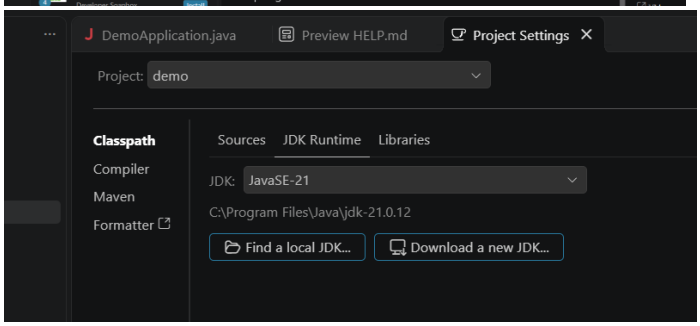
Maven is a build and dependency management tool. For this laboratory, do NOT install global Maven: the generated Spring Boot project will include Maven Wrapper (mvnw.cmd), which downloads/runs the project-specified Maven distribution as needed. Likewise, do NOT install standalone Tomcat: Spring Boot runs an embedded servlet container.

Component	How it becomes available
JDK 21	Install locally on Windows
VS Code extensions	Install via VS Code Extensions
Maven	Run project Maven Wrapper after project generation
Spring Boot and Thymeleaf	Maven downloads project dependencies
Tomcat	Embedded via Spring Web MVC starter
Git	Install separately when reaching Stage H

Screenshots:







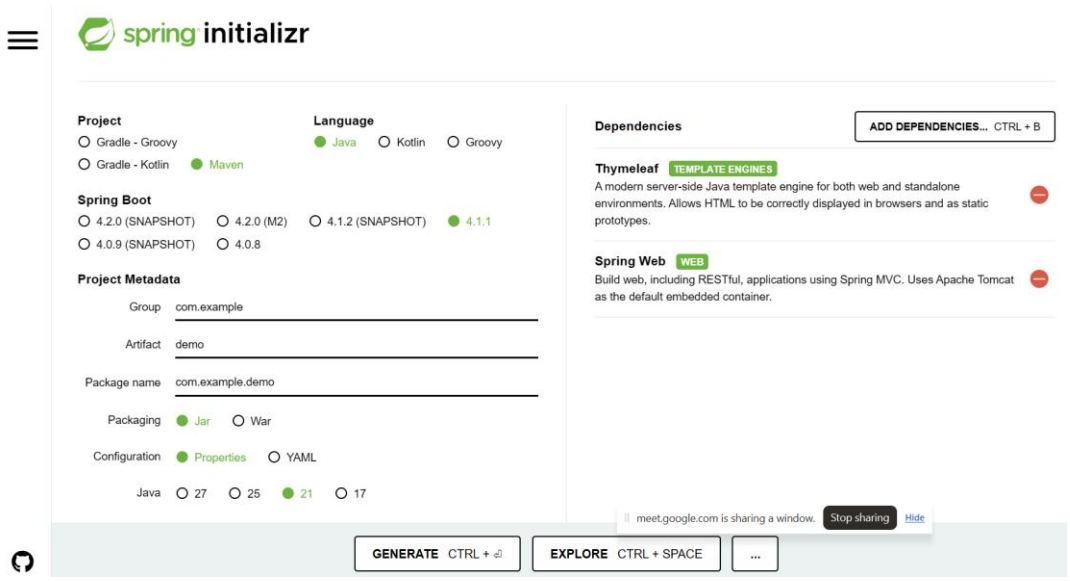
E. Generate and inspect the Spring Boot project

Task E1 – Generate with Spring Initializr

11. Open https://start.spring.io/ in a browser. Choose Maven, Java, a lecturer-approved Spring Boot 4.x version, Group my.utm.SCSE3203, Artifact week02-demo, Package my.utm.SCSE3203.week02, JAR packaging and Java 21.
12. Add “Spring Web MVC” and “Thymeleaf” dependencies. With Spring Boot 4.x, inspect whether the project uses spring-boot-starter-webmvc and spring-boot-starter-thymeleaf.
13. Generate the ZIP, extract to C:\SCSE3203\week02-demo, and open the project ROOT folder in VS Code (the folder containing pom.xml).
14. Wait for Java project import / dependency resolution. Allow Maven Wrapper any permitted first-run downloads.

VERIFY ☐ Project root contains pom.xml, mvnw.cmd, .mvn/, src/main/java and src/main/resources.

IF IT FAILS Do not open only the src folder. If dependency retrieval fails, check internet/proxy and the Maven error output before retrying. The exact Spring Boot patch should be consistent across the class.



Task E2 – Verify Maven Wrapper and POM

```
.\mvnw.cmd -version
```

Expected: Maven version, Java 21, and Windows environment information. It may download a Maven distribution on first run.

VERIFY ☐ Maven Wrapper command completes successfully and shows Java 21.

15. Open pom.xml. Find groupId, artifactId, version and both web MVC and Thymeleaf dependencies. Explain why there are no manual JAR downloads.
16. Locate the main application class and application.properties. Identify templates/ versus static/. Empty directories may not be visible until files are created.

Discussion prompt

Explain why dependency declarations in pom.xml cause additional transitive libraries to be downloaded and why target/ must not be uploaded as source code.

F. Develop the first server-rendered website

Task F1 – Run the generated application

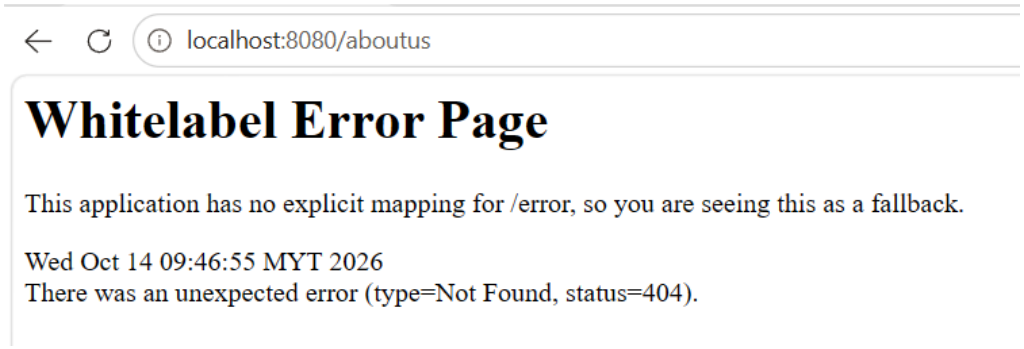
```
.\mvnw.cmd spring-boot:run
```

Watch the console for a successful start. Open http://localhost:8080/ . A 404 at this point may be normal because the root route has not yet been defined.

VERIFY ☐ The server starts on port 8080, even if / currently returns a 404.

IF IT FAILS Build failure: inspect the first meaningful Maven/compiler error. "Port 8080 already in use": stop the conflicting program or choose an available port. To stop the running application use Ctrl+C.

Expected output (web browser)



Means?

- Spring Boot server is reachable, but the requested URL cannot be resolved to an available endpoint or resource (server is up and running, but requested resource/page not found - 404 -).

Task F2 – Create a controller

17. Under src/main/java/my/utm/SCSE3203/week02/, create a controller package and a HomeController.java file.

```
package my.utm.SCSE3203.week02.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {
    @GetMapping("/")
    public String home() {
        return "index";
    }
}
```

Explanation: @Controller declares a Spring MVC page controller; @GetMapping("/") selects the root GET path; return "index" identifies a logical view name, not a plain-text response. The details of MVC internals are covered in a later week.

VERIFY ☐ HomeController compiles, is under the application base package and maps "/".

Task F3 – Create an HTML template

18. Create src/main/resources/templates/index.html, exactly matching the view name index.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>SCSE3203 | Week 02</title>
</head>
<body>
    <header><h1>SCSE3203 – Special Topics</h1></header>
    <main>
        <h2>Welcome to Java Web Development</h2>
        <p>My first Spring Boot server-rendered web page.</p>
        <h3>Week 02 – Development Environment</h3>
        <ul><li>Java 21</li><li>Maven</li><li>Spring Boot</li><li>Thymeleaf</li></ul>
        <p><a href="/about">About the course</a></p>
    </main>
</body>
</html>
```

19. Run or restart the application. Visit http://localhost:8080/ and use View Page Source to inspect the returned HTML.

VERIFY ☐ Homepage displays heading, introductory paragraph and technology list in a browser.

IF IT FAILS 404: check mapping, controller package and application startup. 500/template not found: ensure index.html is in src/main/resources/templates/ and that the Thymeleaf dependency exists. A link to /about will remain unimplemented until Task G1.

G. Extend the site and verify configuration

Task G1 – About page (target in class)

20. Add the following method to HomeController, inside the existing class:

```
@GetMapping("/about")
public String about() {
    return "about";
}
```

21. Create src/main/resources/templates/about.html:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>About SCSE3203</title></head>
<body>
    <h1>About SCSE3203</h1>
    <p>Java web development using Spring Boot and Thymeleaf.</p>
    <a href="/">Back to Home</a>
</body>
</html>
```

22. Test / and /about. Confirm that the Home ↔ About links work in both directions.

VERIFY ☐ Both web pages render correctly and navigation works.

Task G2 – Configure a different port

23. Edit src/main/resources/application.properties and add:

```
spring.application.name=week02-demo
server.port=8081
```

24. Restart Spring Boot and open http://localhost:8081/ . Verify the new port, then restore port 8080 (or delete server.port) for consistency.

VERIFY ☐ Application works on configured port and returns to 8080 after restoration.

Task G3 – Package application (complete outside class if needed)

```
.\mvnw.cmd clean package
```

If the standard generated tests pass, Maven creates a JAR file under target/. Run using the actual JAR filename (example below assumes default version):

```
java -jar target\week02-demo-0.0.1-SNAPSHOT.jar
```

VERIFY ☐ Packaged JAR starts outside the VS Code Run button and serves the homepage.

IF IT FAILS If JAR packaging fails, read build/test errors first. If a server is already running, stop it before launching the JAR on the same port.

H. Install Git and save to GitHub

Task H1 – Git from zero setup

25. Download Git for Windows from https://git-scm.com/downloads/win . Use standard installer choices unless instructed otherwise.
26. Open a new PowerShell/VS Code terminal and verify git --version, then configure your own name/email.

```
git --version
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git config --global --list
```

VERIFY ☐ Git version displayed and commit identity is configured.

Task H2 – Record your work

If using GitHub Classroom, follow the lecturer's invitation and repository instructions. Clone the assigned repository and work within that clone. Do not reinitialise a repository or force-push over supplied content. If using a newly created empty GitHub repository, the following illustrates the normal local workflow.

```
git init
git status
git add .
git commit -m "Create Spring Boot server-rendered course website"
```

Before committing, inspect .gitignore; target/ and secret credentials must not be committed. For an empty remote already created on GitHub, connect it using its actual URL and push. The GitHub sign-in flow may ask for browser-based authentication.

```
git branch -M main
git remote add origin YOUR-EMPTY-REPOSITORY-URL
git push -u origin main
```

VERIFY ☐ GitHub repository contains source files, pom.xml, Maven Wrapper and HTML templates; target/ is not committed.

IF IT FAILS 403/repository not found: check invitation acceptance and repository permissions. Authentication problems: sign in through Git Credential Manager or VS Code. Non-empty remote: do not blindly overwrite it.

I. Common errors — quick diagnosis

Symptom	Check first	Corrective approach
java/javac not recognised	Version + PATH	Restart terminal; verify full JDK and JAVA_HOME
Wrong Java version	where.exe java	Resolve installed JDK selection
VS Code imports unresolved	Java language server + pom.xml	Reload Java project; Maven resolution
Maven dependency download error	Network / logs	Resolve proxy or connectivity; retry
Server 8080 unavailable	Occupied port	Stop process or change server.port
404 on /	Controller mapping / package	Check @Controller and @GetMapping
500 / template error	Template path / dependency	Check templates/index.html and Thymeleaf
GitHub push denied	Authentication and remote URL	Authenticate and verify access

J. Evidence of successful completion

Check	Student status
java -version and javac -version show Java 21	☐
Required VS Code extensions installed	☐
mvnw.cmd -version works	☐
Spring Boot application starts	☐
Home page renders at localhost	☐
About page and navigation work	☐
Application packaged as JAR	☐
Git/GitHub repository verified	☐

Minimum pass during class: environment functional and server-rendered homepage visible. Other targets can be continued as guided SDL if needed.

K. Independent SDL challenge (after class)

Create /profile using @GetMapping and profile.html in templates. The page should use a fictional student profile, provide links to Home and About, and be committed to Git. Do not add REST or JSON endpoints.

Submission evidence (according to lecturer instructions): project repository URL, screenshot of rendered pages, terminal version evidence, and a 100–150-word reflection describing one error and how it was diagnosed.

L. Reflection and exit ticket

27. How do JDK, Maven, Spring Boot, embedded Tomcat and Thymeleaf work together?
28. Why does return "index" render index.html instead of sending the text “index”?
29. What is the difference between a Git commit and a Git push?
30. Which checkpoint was most difficult and what evidence helped you solve it?

Appendix – preparation

- Check your network access to Adoptium, VS Code extensions, Spring Initializr, Maven Central and GitHub.
- Prepare offline installers where possible; Maven may need a prewarmed dependency cache in restricted labs.
- Pin the exact Spring Boot 4.x patch and Java 21 version demonstrated in class; verify starter names using that version.
- Test all commands on a clean Windows user account; ensure Java language services finish importing.

References

https://adoptium.net/

https://code.visualstudio.com/docs/java/java-spring-boot

https://start.spring.io/

https://docs.spring.io/spring-boot/reference/

https://spring.io/guides/gs/serving-web-content

https://maven.apache.org/guides/

https://git-scm.com/doc